Unit 3-Imp-Questions-Solutions

Operating Systems (Dr. A.P.J. Abdul Kalam Technical University)

Scan to open on Studocu

Studocu is not sponsored or endorsed by any college or university
Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

UNIT – 3
CPU Scheduling
PART-1

2 Marks Questions

Q. Why do we need Scheduling? (AKTU 2021-22)

Ans.

Scheduling deals with the problem of deciding which of the processes waiting in the ready queue for processing requests is to be allocated resources. There are many different scheduling algorithms, like FCFS, SJF, Round Robin, Priority based etc.etc.. An operating system uses process scheduling to ensure that processes execute efficiently and have reduced wait times. The goal of process scheduling policies is to use CPU resources wisely, increase throughput, reduce wait time, increase response and turnaround times.

Q. What are the Performance Criteria in CPU Scheduling? (AKTU 2021-22)

OR

Q. What are the various Scheduling Criteria for CPU Scheduling? (AKTU 2018-19) (AKTU 2017-18)

Ans.

CPU Scheduling Criteria

- **CPU Utilization:** It is the criteria of scheduling that no CPU cycle should be wasted. All the times, there should be a process running by CPU. Ideally it should be 100% but in real scenario CPU usage should be in a range of 40% to 90%
- **Throughput**: It is amount of work done in a given unit time. Here in the context of scheduling it is the no of processes completed in a given unit time period.
- **Turnaround Time:** It is the amount of time taken to execute a particular process, i.e. The time interval from time of submission of the process to

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

the time of completion of the process. It is basically completion time of the process minus arrival time of process.

- **Waiting Time:** The amount of time a process has been waiting in the ready queue to get control on the CPU. It can be calculated as turnaround time minus execution time of a process.
- **Response Time:** Interval of time it takes from when a ready request was submitted until the first response is produced. Note that, it is the time till the first response and not the completion of process execution(final response).

In general CPU utilization and Throughput are maximized and other factors are reduced for proper optimization.

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

Q. Describe the typical elements of the Process control block. (AKTU 2018-19)

OR

What is the need for Process Control Block (PCB) (AKTU 2015-16)

Ans.

Operating System maintains a data structure called **P**rocess **C**ontrol **B**lock **(PCB)** for every process when it is created.

A PCB keeps all the information needed to keep track of a process as listed below in the table –

- **Process State:** The current state of the process i.e., whether it is ready, running, waiting, or whatever.
- **Process privileges:** This is required to allow/disallow access to system resources.
- **Process ID:** Unique identification for each of the process in the operating system.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Process ID
Process State
Pointer
Program Counter
Process privileges
CPU registers
Accounting Information
I/O information
Memory mgmt. Information
:
:

- **Pointer:** A pointer to parent process.
- **Program Counter:** Program Counter stores the address of the next instruction to be executed for this process.
- **CPU registers:** Register set where process needs to be stored for execution for running state. They may include accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information.
- **CPU Scheduling Information**: information scheduling CPU time
- **Memory Management Information**: page table, memory limits, segment table
- **Accounting Information**: amount of CPU used for process execution, time limits, execution ID etc.
- **I/O Status Information**: list of I/O devices allocated to the process and the list of open files etc.

PCB is the repository of any information that may vary process to process

---

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Q. Explain Context Switching. AKTU- 2018-19

OR

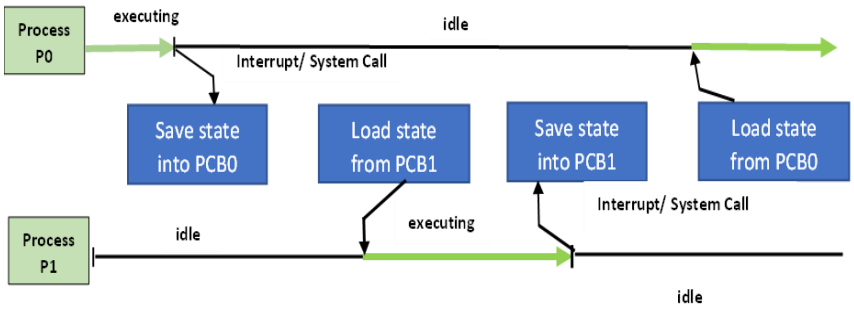
Q. What is the use of Context Switching? AKTU- 2017-18

Ans.

Switching the CPU from one process to another process requires performing a state save of the current process and a state restore of another process. This task is known as a **context switch**.

When a context switch occurs, the **kernel saves the context** of the old process in its PCB and **loads the saved context** of the new process scheduled to run.

Context-switch time is an overhead to the system because it does not do any useful work while switching.



Q. What do you mean by the safe state and an unsafe state? AKTU- 2021-22

OR

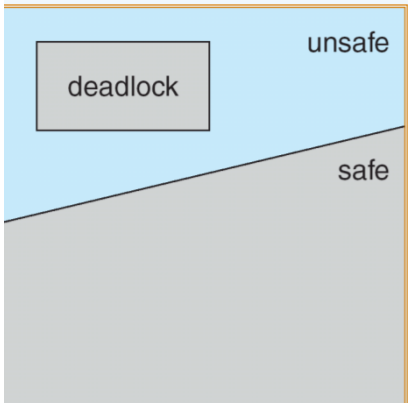
Q. What is safe state and unsafe state? AKTU- 2018-19

Ans.

Safe State

- State is ***safe*** if the system can allocate resources to each process (up to its maximum) in some order and still avoid a deadlock.
- More formally, a system is in a safe state only if there exists a **safe sequence.**

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)



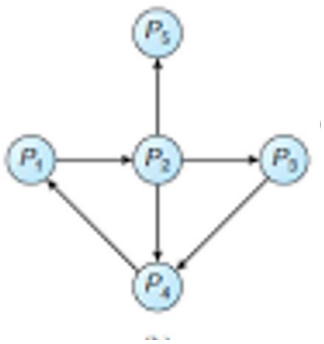
- If there does not exist any order or sequence of processes in which resources can be allocated to them, then it is ***unsafe state*.**
- Unsafe state means that there may be a deadlock but not necessary condition.

Q. Discuss the usage of wait-for graph method AKTU- 2017-18

Ans.

Wait-for graph is a method to detect deadlock condition. If all resources have only a single instance, then we can define a deadlock detection algorithm that uses a variant of the resource-allocation graph, called a wait-for graph.

A deadlock exists in the system if and only if the wait-for graph contains a cycle.



Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

10 Marks Questions

Q. 1. Define Process. Explain various steps involved in change of a process state with neat transition diagram. **AKTU- 2018-19**

OR

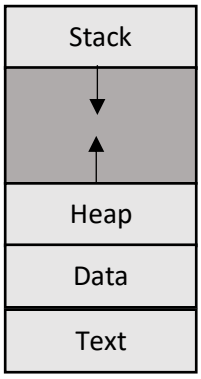
What do you understand by Process? Explain various states of process with suitable diagram. Explain process control block.

AKTU- 2016-17, MARKS 10

Answer:

Process Concept

- Process is a program in execution.
- process will need certain resources—such as CPU time, memory, files, and I/O devices to accomplish its task.
- When a program is loaded into the memory and it becomes a process, it can be divided into four sections **— stack, heap, text and data**. The image shows a simplified layout of a process inside main memory

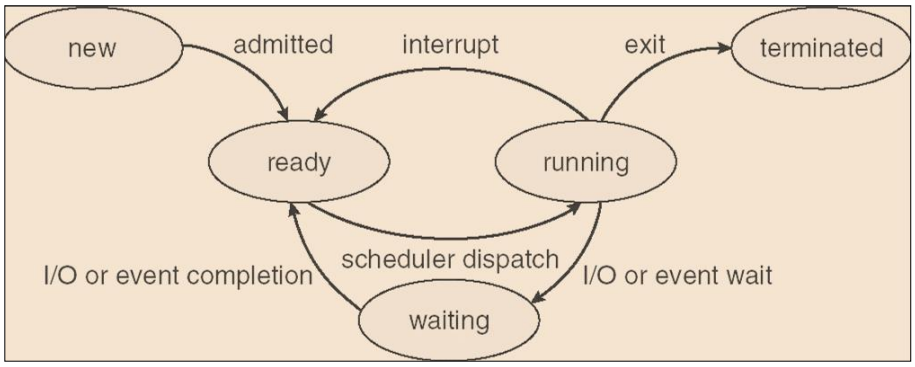


A Process in

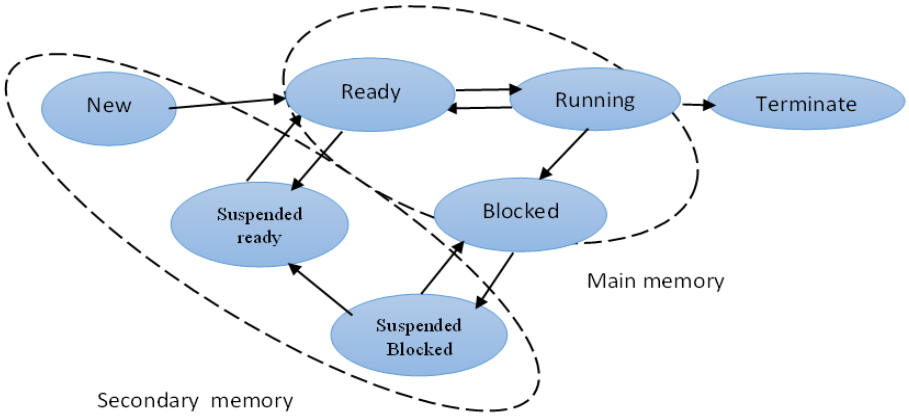
Process States or Process Life Cycle

- There is process life cycle, where it passes through different stages. These stages may differ in different operating systems.
- In general, when a process executes it can have one of the following five states in its life cycle.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)



- **New (Create):** It is the program, which is present in secondary memory that will be selected by OS to create the process.
- **Ready:** Ready to execute. After a process is created, the process enters the ready state i.e. the process is loaded into the main memory. The process here is ready to run and is waiting to get the CPU time for its execution.
- **Running:** The process gets CPU for execution and all its instructions are run one after another.
- **Waiting**: Whenever the process requests access to I/O for taking input from the user or for output to device, it enters the blocked or wait state.
    - The process continues to wait in the main memory and does not require CPU.
    - Once the I/O operation is completed the process goes to the ready state.
- **Terminated or completed**: Process is terminated when it is completed or it is killed for any reason. When a process is terminated its PCB is also deleted.
- Sometimes a process can have seven states also. Apart from the above five states it can have additional two states i.e. **Suspended Blocked and Suspended Ready state.**



Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- **Suspend ready:** Process that was initially in the ready state but were swapped out of main memory and placed onto external storage by scheduler are said to be in suspend ready state. The process will again go to ready state whenever the process is again brought onto the main memory.
- **Suspend wait or Suspend Blocked** Similar to suspend ready but uses the process which was performing I/O operation and due to lack of main memory swapped out to secondary memory. When I/O operation is completed it may go to suspend ready.
- **Some of the reasons to suspend a process are...**
  1.If one process is ready to execute, but there is no space in the main memory, then it is suspended.
  2. When one process in main memory which was blocked & there is another process ready to execute, but waiting in secondary memory, then the process in main memory is suspended.
  3. When the parent process suspends, then the sub process is also suspended.
-

Process Control Block

Operating System maintains a data structure called **P**rocess **C**ontrol **B**lock **(PCB)** for every process when it is created. It contains all the information about a process. In some operating systems the PCB is placed at the bottom of the process stack.

A PCB keeps all the information needed to keep track of a process as listed below in the table –

Process ID
Process State
Pointer
Program Counter
Process privileges
CPU registers
Accounting Information
I/O information
Memory mgmt. Information
:
:

- **Process State:** The current state of the process i.e., whether it is ready, running, waiting, or whatever.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- **Process privileges:** This is required to allow/disallow access to system resources.
- **Process ID:** Unique identification for each of the process in the operating system.
- **Pointer:** A pointer to parent process.
- **Program Counter:** Program Counter stores the address of the next instruction to be executed for this process.
- **CPU registers:** Register set where process needs to be stored for execution for running state. They may include accumulators, index registers, stack pointers, and general-purpose registers, plus any condition-code information.
- **CPU Scheduling Information**: information scheduling CPU time
- **Memory Management Information**: page table, memory limits, segment table
- **Accounting Information**: amount of CPU used for process execution, time limits, execution ID etc.
- **I/O Status Information**: list of I/O devices allocated to the process and the list of open files etc.

PCB is the repository of any information that may vary process to process

---

Q. 2

What is the Process Control Block. Discuss differences between short term, medium term and long term schedulers. (AKTU 2015-16, MARKS 10)

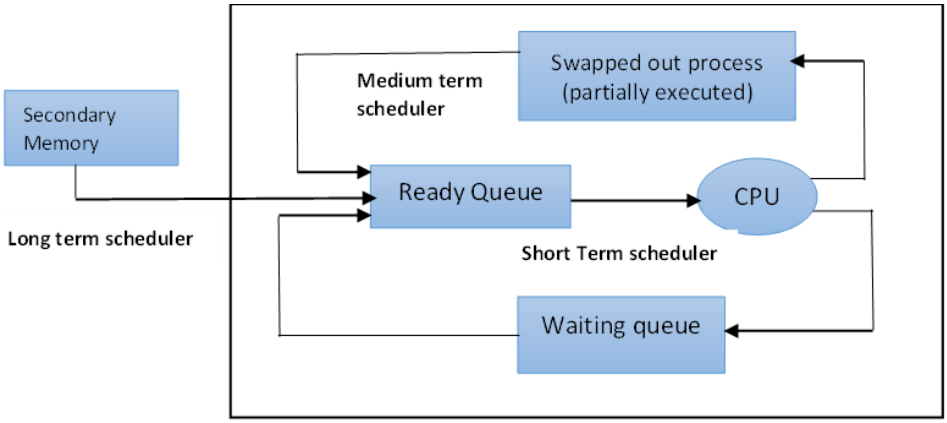
Answer:
PCB is explained in Q.1. above
Schedulers

There are different schedulers which selects processes to move them from one queue to another queue. Normally there are three schedulers.

1. **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue (from the secondary memory to main memory).
2. **Short-term scheduler** (or **CPU scheduler**) – selects a process from the ready queue, which should be executed next and allocates CPU.
3. **Medium-term scheduler** . The process is swapped out, and is later swapped in, by the medium-term scheduler.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- The key idea behind a medium-term scheduler is that sometimes a process can be removed from main memory (and from active contention for the CPU) and sent to secondary memory due to requirement of main memory by other active process.
- Later, the process can be again brought into main memory, and its execution can be continued where it left off.
- This scheme is called **swapping**.
- The process is swapped out, and is later swapped in, by the medium-term scheduler.
- Swapping may be necessary to improve the process mix or because a change in memory requirements has overcommitted available memory, requiring memory to be freed up.



Q. 3.

What is CPU scheduling? Give the criteria for scheduling.

OR

Discuss the performance criteria for CPU scheduling.

AKTU 2015-16, Marks 10

Answer:

CPU Scheduling

- In a single-processor system, only one process can run at a time. Other processes have to wait until the CPU is free. The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization. When a process is executed and then for some I/O event wait for the completion of the event, the CPU then just sits idle, no useful work is accomplished.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- Several processes are kept in memory at one time as ready queue.
- When one process has to wait, the operating system takes the CPU away from that process and gives the CPU to another process.
- The selection of a process from the ready queue to be moved to CPU to run is called **process/ CPU scheduling.**

CPU scheduling is of two types:

1. Non Pre-emptive Scheduling
2. Pre-emptive Scheduling

Non Pre-emptive Scheduling

- In non-pre-emptive scheduling, once the scheduler allocates the CPU to a process, the process keeps the CPU until it switches to the waiting state, as the result of an I/O request or an invocation of wait(), or when process terminates.

Pre-emptive Scheduling

- In this type of Scheduling, the running task can be interrupted for some time due to tasks with higher priority has come, and resumed later when the priority task has finished its execution.

CPU Scheduling Criteria

- **CPU Utilization:** It is the criteria of scheduling that no CPU cycle should be wasted. All the times, there should be a process running by CPU. Ideally it should be 100% but in real scenario CPU usage should be in a range of 40% to 90%
- **Throughput**: It is amount of work done in a given unit time. Here in the context of scheduling it is the no of processes completed in a given unit time period.
- **Turnaround Time:** It is the amount of time taken to execute a particular process, i.e. The time interval from time of submission of the process to the time of completion of the process. It is basically completion time of the process minus arrival time of process.
- **Waiting Time:** The amount of time a process has been waiting in the ready queue to get control on the CPU. It can be calculated as turnaround time minus execution time of a process.
- **Response Time:** Interval of time it takes from when a ready request was submitted until the first response is produced. Note that, it is the time till the first response and not the completion of process execution(final response).

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Optimization Criteria

In general CPU utilization and Throughput are maximized and other factors are reduced for proper optimization.

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

Q. 4.

What is the difference between pre-emptive and non-pre-emptive scheduling? State, why strict non pre-emptive scheduling is unlikely to be used in computer centre? Explain the operation of multi-level scheduling.

Answer:

Preemptive Scheduling	Non-Pre-emptive Scheduling
The CPU is allocated to the processes for the limited time.	The CPU is allocated to the process till it terminates or switches to waiting state
The executing process in preemptive scheduling is interrupted in the middle of execution	The executing process in non-pre-emptive scheduling is not interrupted in the middle of execution.
Preemptive Scheduling has the overhead of switching the process from ready state to running state, vice-versa, and maintaining the ready queue	Non-pre-emptive scheduling has no overhead of switching the process from running state to ready state.
In preemptive scheduling, if a process with high priority frequently arrives in the ready queues then the process with low priority has to wait for a long time, and it may have to starve.	The non-pre-emptive scheduling, if CPU is allocated to the process with larger burst time then the processes with small burst time may have to starve.
Preemptive scheduling is quite flexible because the critical processes are allowed to access the CPU as they arrive into the ready	Non-pre-emptive scheduling is rigid as even if a critical process enters the ready queue the process running the CPU is not disturbed.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

queue, no matter what process is executing currently.	
The Preemptive Scheduling is cost associative as it has to maintain the integrity of shared data	The non-Pre-emptive Scheduling is not cost associative

Q. 5.

Describe process address space.

Answer:

1. Process address space means a space that is allocated in memory for a process.
2. Address space is a space in computer memory.
3. Every process has an address space.
4. Address space can be of two types:

Physical address space: Physical address space is created in RAM.
Virtual address space: Virtual address space is an address space that is created outside the main memory inside the virtual memory, and it is created in hard disk.

Q. 6.

Write a short note on process identification information.

Answer:

1. Process registration involves recording all information necessary to identify a registered process and to differentiate it from other processes in the system. This information is called process identification information.
2. This information is recorded right after the process is created or born.
3. Usually, every process is also given a unique identifier, called a process ID, in order to make future references easier and unambiguous.
4. Process identification information is usually static, which means it does not change as time goes by and as the process moves from one state of life to another, for that purpose, the system will record process state information.
5. Process identification information use following numeric identifiers :

a. Unique process identifier (PID) : PID provide index id to the process.
b. User identifier (UID): UID identifies the user who is responsible for the job.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

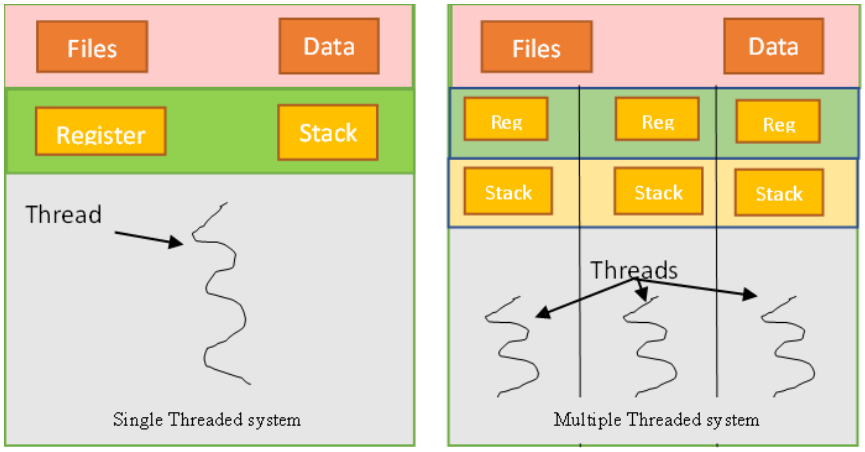
Q. 7

Explain in detail about the Threads and their management.
(AKTU 2021-22)

Answer:

Threads and their Management

- When a process can be split into many sub-processes which can be scheduled and executed at the same time, they are called **threads**.
- There can be more than one thread inside a process, but all these threads belong to one process only. No thread can exist outside the process.
- A thread is a basic unit of CPU utilization, which consists of a thread-id, program counter, registers and a stack.
- All Threads of a process, share their code section, data section other operating system details such and files.
- Therefore, thread is often referred to as a **lightweight process**.



- An application is implemented as a separate process with several threads of control running at the same time.
- For example a web browser is busy with displaying text with on thread and simultaneously it can display images with another thread, and even can search data or web page with yet another thread.
- If system is multithreaded, these all requests can be handled all together with **quick response time.**

Advantage of Threads

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- **Responsiveness:** An application may allow a program to continue running even if it's one thread is blocked for I/O operation
- **Resource sharing:** Threads share the memory and other resources like code and data of the process to which they belong. This allows an application to have several different threads of activity within the same address space.
- **Cost of Execution:** Since threads are sharing resources of the process, it is more economical to switch between threads and creating new threads because they are working in same address space.
- **Efficient communication:** Threads of a process can communicate with one another through shared address space, thus avoiding the overhead of the system calls for communication.
- **Scalability of Parallelism:** A single-threaded process can run on only one processor, whereas in multithreading in a multiprocessor architecture, threads may be running in parallel on different processing cores.

Q. 8.

How threads are implemented? Explain types of threads.

OR

What are the types of thread? Give advantages and disadvantages.

Answer:

- A thread library provides programmers with an API for creating and managing threads. Support for threads must be provided either at the user level or by the kernel.

Multithreading Models

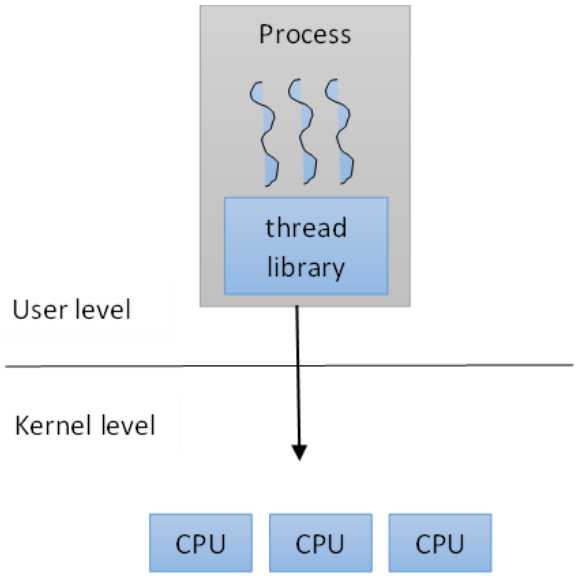
- A relationship exist between user threads and kernel threads.
- There are three types relationship models of user and kernel threads
  - The many-to-one model,
  - The one-to-one model, and
  - The many-to-many model

Types of Threads

- **User Level Threads**
  - User threads are supported above the kernel and are managed without kernel support
  - In this case, the thread management kernel is not aware of the existence of threads.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- The thread library contains code for creating and destroying threads, for passing message and data between threads, for scheduling thread execution and for saving and restoring thread contexts.



Advantages

- Thread switching does not require Kernel mode privileges and switching is fast.
- User level thread can run on any operating system, even that doesn't support multithreading.
- User level threads are fast to create and manage.

Disadvantages

- No or less co-ordination among the threads and Kernel.
- If one thread causes a page fault, the entire process blocks.
- In a typical operating system, most system calls are blocking.
- Multithreaded application cannot take advantage of multiprocessing.

Kernel Level Threads

- Thread management is done by the Kernel in kernel level threads.
- Kernel threads are supported directly by the operating system.
- The Kernel maintains context information for the process as a whole and for individuals threads within the process.
- Scheduling by the Kernel is done on a thread basis.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- The Kernel performs thread creation, scheduling and management in Kernel space.

Advantages

- Kernel can simultaneously schedule multiple threads from the same process on multiple processors.
- If one thread in a process is blocked, the Kernel can schedule another thread of the same process.
- Kernel routines themselves can be multithreaded.

Disadvantages

- Kernel threads are generally slower to create and manage than the user threads.
- Transfer of control from one thread to another within the same process requires a mode switch to the Kernel.

Q. 9.

Differentiate between user level and kernel level thread.

AKTU 2012-13, Marks 2.5

Differentiate between user thread and kernel thread. What is thread cancellation?

AKTU 2013-14, Marks 05

Answer:

User –Level Thread	**Kernel-Level Thread**
User-level threads are faster to create and manage	Kernel-level threads are slower to create and manage.
Implementation is by a thread library at the user level.	Operating system supports creation of Kernel threads.
User-level thread is generic and can run on any operating system.	Kernel-level thread is specific to the operating system.
Multi-threaded applications cannot take advantage of multiprocessing	Kernel routines themselves can be multithreaded.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Q 3.10

What is thread? How thread is different from a process? What resources are used when a thread is created?

AKTU 2014-15, Marks 10

Answer:

- When a process can be split into many sub-processes which can be scheduled and executed at the same time, they are called **threads**.
- There can be more than one thread inside a process, but all these threads belong to one process only. No thread can exist outside the process.
- A thread is a basic unit of CPU utilization, which consists of a thread-id, program counter, registers and a stack.
- All Threads of a process, share their code section, data section other operating system details such and files.
- Therefore thread is often referred to as a **lightweight process**.

Difference between thread and process:

Parameters	**Thread**	**Process**
Definition	Thread means a segment of a process	Process means a program is in execution.
Lightweight	Threads are Lightweight.	The process is not Lightweight.
Termination Time	The thread takes less time to terminate	The process takes more time to terminate.
Creation Time	It takes less time for creation.	It takes more time for creation.
Communication	Communication between threads requires less time compared to processes.	Communication between processes needs more time compared to thread.
Context Switching Time	It takes less time for context switching	It takes more time for context switching
Resource	Thread consumes fewer resources.	Processes consume more resources.
Treatment By OS	All the level peer threads are treated as a single task by OS.	Different process are treated separately by OS
Memory	Threads share memory.	The process is mostly isolated
Sharing	Threads share data with each other.	It does not share data

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Thread cancellation allows a thread to terminate the execution of any other thread in the process. The target thread (the one being cancelled) can keep cancellation requests pending and can perform application-specific cleanup when it acts upon the cancellation notice.

Q 3.11

Discuss First Come First Serve (FCFS) scheduling algorithm. Give its advantages and disadvantages.

Answer:

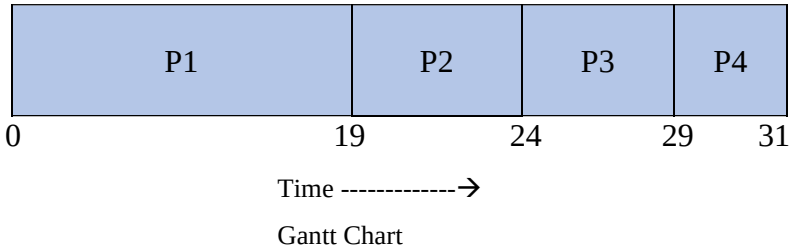
First Come First Serve (FCFS)

- As the name suggests, in the "First Come first Serve" scheduling algorithm, the process which comes in ready queue first is executed first by the CPU.
- It is very simple to implement with 'Queue' data structure of in which insertion is done at the rear end and deletion is done from the front. (In front is the first process to arrive and in rear is the last process to arrive in the ready queue
- This is **used in Batch Systems**.
- A perfect real life example of FCFS scheduling is booking tickets at ticket counter of Movie etc.

Consider the processes P1, P2, P3, P4 given in the below table, arrives for execution in the same order, with Arrival Time 0, and given Burst Time in milliseconds, let's find the average waiting time and average turnaround time using the FCFS scheduling algorithm.

Process Queue	Arrival Time AT	Burst Time BT	Turnaround Time (CT – AT)	Waiting time (TAT – BT)	Response Time
P1	0	19	19	0	0
P2	0	5	24	19	19
P3	0	5	29	24	24
P4	0	2	31	29	29
		Average	103/4= 25.75	72/4= 18	72/4 =18

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)



Drawbacks of FCFS

- It is **Non Pre-emptive** algorithm, which means the **process priority** or **short burst time** process doesn't matter. Suppose a long process (backup process) comes before a short and high priority process (interrupt), then short process or, the high priority process will have to wait, and hence in this case, the system may crash, just because of non-preemptive scheduling.
- Not optimal Average Waiting Time.
- Resources utilization in parallel is not possible, which leads to **Convoy Effect**, and hence poor resource(CPU, I/O etc) utilization.
- **Convoy Effect** is a situation where many processes, who need to use a resource for short time are blocked by one process holding that resource for a long time.

Que 3.12

Discuss Shortest Job First (SJF) scheduling algorithm. Also, write its advantages and disadvantages.

Answer:

Shortest Job First(SJF) Scheduling

- In this scheduling algorithm the ready queue is checked for the shortest burst time process.
- This is the best approach to minimize waiting time.
- This also can be used in Batch Systems only when all process have same arrival time.
- It is of two types:
  - **Non Pre-emptive SJF**
  - **Pre-emptive (S**hortest **R**emaining **T**ime **F**irst**)**
- The choice arises when a new process arrives at the ready queue, with shorter CPU burst than a previous process which is currently executing.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- A **pre-emptive** SJF algorithm will pre-empt the currently executing process, whereas a **non-preemptive** SJF algorithm will allow the currently running process to finish its CPU burst.
- It can be successfully implement if, the burst time/execution time of the processes is known to the processor in advance, which is practically not feasible all the time.
- This scheduling algorithm is optimal if all the jobs/processes are available at the same time i.e. either arrival time is 0 for all, or arrival time is same for all

- Shortest Job first has the advantage of having a minimum average waiting time among all scheduling algorithms.
- It is a Greedy Algorithm.

Advantages -

- Shortest jobs are favored.
- It is provably optimal, in that it gives the minimum average waiting time for a given set of processes.

Disadvantages –

- SJF may cause starvation, if shorter processes keep coming. This problem is solved by aging.
- It cannot be implemented at the level of short term CPU scheduling

Que 3.13

Discuss priority scheduling algorithm. What are its advantages and disadvantages?

Answer:

Priority scheduling can be of two types:

1. **Preemptive Priority Scheduling**: If the new process arrived at the ready queue has a higher priority than the currently running process, the CPU is preempted, which means the processing of the current process is stopped and the incoming new process with higher priority gets the CPU for its execution.
2. **Non-Preemptive Priority Scheduling**: In case of non-preemptive priority scheduling algorithm if a new process arrives with a higher

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

priority than the current running process, the incoming process is put at the head of the ready queue, which means after the execution of the current process it will be processed.

Advantages of Priority Scheduling

- The priority of process is selected on the basis of memory requirement, user preference or the requirement of time.
- Processes are executed on the basis of priority. So high priority does not need to wait for long which saves time.
- It is easy to use
- It is a user friendly algorithm.
- Simple to understand
- It has reasonable support for priority

Disadvantages of Priority Scheduling

- The major disadvantage of priority scheduling is the process of indefinite blocking or starvation. This problem appears when a process is ready to be executed but it has to wait for the long time for execution by CPU because other high priority processes are executed by the CPU.
- In case if we have the processes which have the same priority, then we have to make use of 2nd scheduling algorithm.
- If the system gets crashes eventually, then all the processes having low priority not finished yet, also get lost.

Que 3.14

Discuss Round Robin scheduling algorithm. Give its advantages and disadvantages.

Answer:

- The **Round-Robin (RR)** scheduling algorithm is designed for interactive environment especially for timesharing systems.
- Primary objective is to provide good response time, and in general to share system resources equitably by all users.
- It is similar to FCFS scheduling, but for time sharing equal time slice or time quantum is given to all processes to execute. After each time slice if process is not complete then CPU is pre-empted for next process to execute.
- A time quantum is generally from 10 to 100 milliseconds in length.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- To implement RR scheduling, the ready queue is treated as a circular queue. New processes are added to the tail of the ready queue.
- The CPU scheduler picks the first process from the ready queue, sets a timer to interrupt after 1 time quantum, and dispatches the process.

Advantages

- Each process is executed equitably by the CPU, so all processes are given the same priority.
- Starvation doesn't occur because for each round robin cycle, every process is given a fixed time to execute. No process is left behind.

Disadvantages

- The performance of the RR algorithm depends heavily on the size of the time quantum.
- At one extreme, if the time quantum is extremely large, the RR policy is the same as the FCFS policy.
- In contrast, if the time quantum is extremely small (say, 1 millisecond), the RR approach can result in a large number of context switches.

Que 3.15

What is CPU scheduling? Discuss the multilevel feedback queue scheduling.

AKTU 2012-13, Marks 10

OR

Explain the functioning of multilevel feedback queue scheduling.

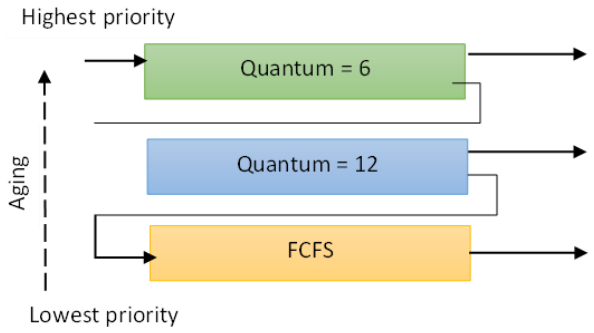
AKTU 2013-14, Marks 10

Answer:

Multilevel Feedback Queue Scheduling

- This type of scheduling allows a process to move between queues.
- The processes are separated according to the CPU time.
- If a process uses too much CPU time, for long time it cannot be held the same queue, it will be moved to a lower-priority queue.
- This scheme leaves I/O-bound and interactive processes in the higher-priority queues.
- In addition, a process that waits too long in a lower-priority queue may be moved to a higher-priority queue.
- This system in which priority of a process can be changed with the time, is called **aging** and it can prevent starvation.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)



- This scheduling algorithm gives highest priority to any process with a CPU burst of 6 milliseconds or less. Such a process will quickly get the CPU, finish its CPU burst, and go off to its next I/O burst. Processes that need more than 6 but less than 12 milliseconds are also served quickly, although with lower priority than shorter processes. Long processes automatically sink to queue 2 and are served in FCFS order with any CPU cycles left over from queues 0 and 1.
- **In general, a multilevel feedback queue scheduler is defined by the following parameters:**
  - The number of queues.
  - The scheduling algorithm for each queue.
  - The method which determines when to move process to a higher priority queue.
  - The method which determines when to move process to a lower priority queue.
  - The method used to determine which queue a process will enter when that process needs service

Advantages

- The scheduling is flexible and allows a process to move between queues.
- A process that waits too long in a lower priority queue may be moved to a higher priority queue which helps in preventing **starvation**.

Disadvantages

- Most complex scheduling algorithm.
- More CPU overheads for a process to move from one queue to another.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Que 3.16

Explain the following scheduling algorithm:

I. Multilevel feedback queue scheduling

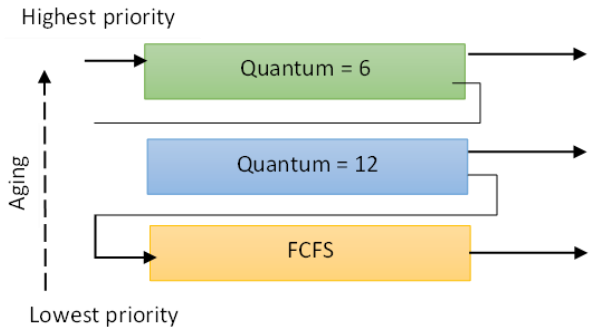
II. Multiprocessor scheduling

AKTU 2014-15, Marks 10

Answer:

Multilevel Feedback Queue Scheduling

- This type of scheduling allows a process to move between queues.
- The processes are separated according to the CPU time.
- If a process uses too much CPU time, for long time it cannot be held the same queue, it will be moved to a lower-priority queue.
- This scheme leaves I/O-bound and interactive processes in the higher-priority queues.
- In addition, a process that waits too long in a lower-priority queue may be moved to a higher-priority queue.
- This system in which priority of a process can be changed with the time, is called **aging** and it can prevent starvation.



- This scheduling algorithm gives highest priority to any process with a CPU burst of 6 milliseconds or less. Such a process will quickly get the CPU, finish its CPU burst, and go off to its next I/O burst. Processes that need more than 6 but less than 12 milliseconds are also served quickly, although with lower priority than shorter processes. Long processes automatically sink to queue 2 and are served in FCFS order with any CPU cycles left over from queues 0 and 1.
- **In general, a multilevel feedback queue scheduler is defined by the following parameters:**
  - The number of queues.
  - The scheduling algorithm for each queue.
  - The method which determines when to move process to a higher priority queue.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- The method which determines when to move process to a lower priority queue.
- The method used to determine which queue a process will enter when that process needs service

Advantages

- The scheduling is flexible and allows a process to move between queues.
- A process that waits too long in a lower priority queue may be moved to a higher priority queue which helps in preventing **starvation**.

Disadvantages

- Most complex scheduling algorithm.
- More CPU overheads for a process to move from one queue to another.

II. Multiprocessor scheduling

- If multiple processors are there then different processes can be executed in parallel by different CPUs.
- Load sharing is possible by different processors but scheduling is complex.
- Scheduler must decide which process to send to which CPU.
- When multiple CPUs in the system share memory, and other peripheral devices with common bus. This system is called **tightly coupled**.
- These systems are mainly used in satellite, weather forecasting, etc. where we want to process bulk amount of data.
- When all processors in the system are independent of each other they are called **loosely coupled system**. Each processor in the system has its own memory and I/O channels.

Different Approaches to Multi-Processor Scheduling

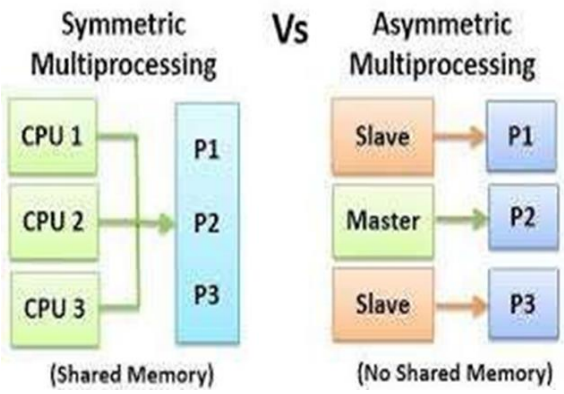
Asymmetric multiprocessing (ASMP)

- When all types of scheduling decisions, including I/O processing, and other system activities are handled by a single processor, the **Master Server**.
- The other processors execute only user code.
- This **ASM** is simple and since one processor accesses the system data structures, it does not need data sharing.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Symmetric multiprocessing (SMP)

- In this approach each processor is capable of self-scheduling.
- All processes may use a common ready queue, or each processor may have its own ready queue of processes.
- The scheduler may have **some issues**, like it must ensure that two separate processors do not choose the same process and processes are not lost from the queue.
- In SMP the load balancing must be handled carefully so that workload is evenly distributed across all processors. Situation like one processor is sitting idle and one process is overloaded, should be avoided.
- Mostly all modern operating systems support SMP, like Windows, Linux, and Mac OS.



Que 3.17

Consider the set of the processes given in the table and the following scheduling algorithm:

I. Round Robin (Quantum = 1)
II. Round Robin (Quantum = 2)
III. Shortest Remaining Job First

Process id	Arrival Time	Execution Time
A	0	4
B	2	7
C	3	3
D	3.5	3
E	4	5

If there is tie within the processes, the tie is broken in the favor of the oldest process. Draw the Gantt chart and find the average waiting time, response time and turnaround time for the algorithms. Comment on your result. Which one is better and why? AKTU 2013-14, Marks **10**

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Answer:

Process id	Arrival Time AT	Execution Time BT	Completion Time CT	Turnaround time TAT (CT – AT)	Waiting time TAT- BT
A	0	4	12	12	8
B	2	7	23	21	14
C	3	3	14	11	8
D	3.5	3	15	11.5	7.5
E	4	5	22	18	13

i) Round Robin (Quantum = 1)

A	A	B	C	D	E	A	B	C	D	E	A	B	C	D	E	B	E	B	E	B	E	B

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23

Average Turnaround Time $= \frac{12 + 21 + 11 + 11.5 + 18}{5} = 14.7$

Avg. Waiting Time = (8 + 14 + 8 + 7.5 + 13) / 5 = 10.1

Response time for Process

A = 0

B = 0

C = 0

D = 4 - 3.5 = 0.5

E = 5 - 1 = 4

Average response Time $= \frac{0.5 + 4}{5} = 0.9$

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

ii) Round Robin (Quantum = 2)

A	B	C	D	E	A	B	C	D	E	B	E	B

0 2 4 6 8 10 12 14 15 16 18 20 21 22

Waiting time for process,

A = 8

B = 2 + 8 + 6 + 2 = 18

C = 4 + 8 = 12

D = 8 + 8 = 16

E = 8 + 8 + 2 = 18

Average Waiting time = $\frac{8 + 18 + 12 + 16 + 18}{5}$ = 14.4

Turnaround time for process,

A = 12

B = 20

C = 11

D = 12.5

E = 17

Average Turnaround time = $\frac{12 + 20 + 11 + 12.5 + 17}{5}$ = 14.5

Response time for process,

A = 0

B = 0

C = 4 - 3 = 1

D = 6 - 3.5 = 2.5

E = 8 - 4 = 4

Average Response time = $\frac{7.5}{5}$ = 1.5

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

(iii) Shortest Remaining Job First

A	A	A	C	D	E	B

0 2 3 4 7 10 15 22

Waiting time for process,

A = 0

B = 15

C = 4

D = 7

E = 10

Average waiting time $= \frac{0+15+4+7+10}{5} = 7.2$

Turnaround time for process,

A = 4

B = 20

C = 4

D = 6.5

E = 6

Average Turnaround time $= \frac{4+20+4+6.5+6}{5} = 8.1$

Response time for process

A = 0

B = 15 - 2 = 13

C = 4 - 3 = 1

D = 7 - 3.5

E = 10 - 4 = 6

Average Response time $= \frac{23.5}{5} = 4.7$

SJF gives minimum average waiting time as compared to Round Robin algorithm. So, it is the best scheduling algorithm

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Que 3.18

Consider the processes; CPU burst time and arrival time given below:

AKTU 2015-16, Marks 10

Processes	CPU burst time	Arrival time
P1	8	0
P2	4	1
P3	9	2
P4	5	3

Draw the Gantt chart and calculate the following by using SRTN CPU scheduling algorithm,

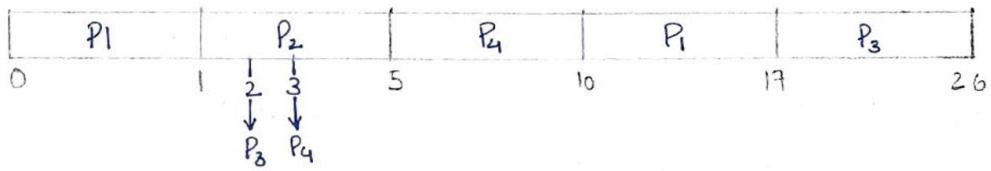
I. Average waiting time

II. Average turnaround time

Answer:

Processes	CPU burst time	Arrival time AT	Completion Time CT	Turnaround time TAT (CT – AT)	Waiting time TAT- BT
P1	8	0	17	17	9
P2	4	1	5	4	0
P3	9	2	26	24	15
P4	5	3	10	7	2

Gantt chart :



Average turnaround time $= \frac{17+4+24+7}{4} = \frac{52}{4} = 13$

Average waiting time $= \frac{9+0+15+2}{4} = \frac{26}{4} = 6.5$

Q 3.

List various performance criteria for scheduling algorithms. Five processes A, B, C, D, E require CPU burst of 3, 5, 2, 5 and 5 units respectively. Their arrival times in the system are 0, 1, 3, 9 and 12 respectively. Draw Gantt chart and compute the average turnaround time and average waiting time of these processes for the Shortest Job First **(SJF)** and Shortest Remaining Time First **(SRTF)** scheduling algorithms. AKTU 2011-12, Marks 10

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Answer:

SJF

Processes	CPU burst time	Arrival time AT	Completion Time CT	Turnaround time TAT (CT – AT)	Waiting time TAT- BT
A	3	0	3	3	0
B	5	1	10	9	4
C	2	3	5	2	0
D	5	9	15	6	1
E	5	12	20	8	3

A	C	B	D	E

0 3 5 10 15 20

Avg. Turnaround time = 3 + 9 + 2 + 6 + 8 = (28/5) = 5.6

Avg. Waiting time = 0 + 4 + 0 + 1 + 3 = (8/5) = 1.6

SRTF

A	C	B	D	E

0 1 2 3 5 9 10 15 20

Avg. Turnaround time = 3 + 9 + 2 + 6 + 8 = (28/5) = 5.6

Avg. Waiting time = 0 + 4 + 0 + 1 + 3 = (8/5) = 1.6

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Que 3.20

Explain in detail about the Deadlock System model and Deadlock characterization. AKTU 2021-22

OR

What is deadlock? Discuss the necessary conditions for deadlock with examples.

AKTU 2016-17

OR

What are the necessary conditions to hold a deadlock in a system?

AKTU 2013-14

Define deadlock. List four necessary conditions for occurrence of deadlock. A system contains 6 units of resource, and the processes that use the resource. What is the maximum value of it for which the system will be deadlock free if the maximum requirement of each process is 3?

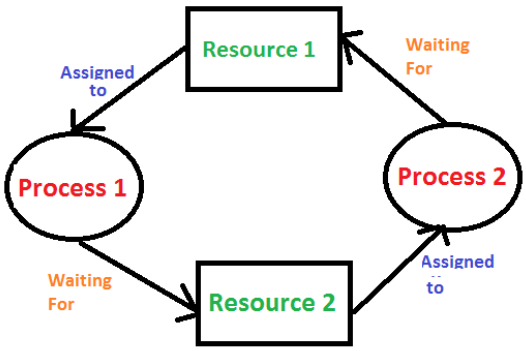
AKTU 2012-13

Answer:

Deadlock

- In a multiprogramming environment, several processes may compete for a finite number of resources.
- A process requests resources; if the resources are not available at that time, because the resources it has requested are held by other waiting processes.
- This situation is called a **deadlock.**

- ***Deadlock*** is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.



Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

There are four necessary conditions that result in Deadlock.

Deadlock can arise if the following four conditions hold simultaneously (Necessary Conditions).

- ***Mutual Exclusion:*** One or more than one resource are non-shareable (Only one process can use at a time)
- ***Hold and Wait:*** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
- ***No Preemption:*** Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- ***Circular Wait:*** A set of processes are waiting for each other in circular form.

Methods for Handling Deadlocks

- we can deal with the deadlock problem in one of three ways:
  - We can use a protocol to **prevent** or **avoid deadlocks**, ensuring that the system will never enter a deadlocked state.
  - We can allow the system to enter a deadlocked state, **detect it, and recover**.
  - We can ignore the problem altogether and pretend that deadlocks never occur in the system.

  - **Deadlock prevention** provides a set of methods to ensure that at least one of the necessary conditions cannot hold.
  - These methods prevent deadlocks by constraining how requests for resources can be made.
  - **Deadlock avoidance** requires that the operating system be given additional information in advance concerning which resources a process will request and use during its lifetime.
  - With this additional knowledge, the operating system can decide for each request whether or not the process should wait.

Maximum value of n:

The maximum value of ‘n’ for which the system is guaranteed to be deadlock free is 2.

Two processes can never lead to deadlock as the peak time demands of(3+3=6) resources can be satisfied. But 3 processes can lead to deadlock if each process holds 2 resources and then demands one more.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Que 3.21

How deadlock are prevented?

OR

Define and explain the deadlock prevention.

OR

What are the approaches that can be used for prevention of deadlock?

AKTU 2013-14, Marks 05

Answer:

Deadlock Prevention

Deadlock prevention is an approach which ensures that system will never enter in deadlock state.

By ensuring that at least one of the necessary conditions cannot hold, we can prevent the occurrence of a deadlock.

- **Mutual Exclusion**: The mutual exclusion condition must hold. That is, at least one resource must be **non-sharable.**
- Sharable resources, in contrast, do not require mutually exclusive access and thus cannot be involved in a deadlock.
- Read-only files are a good example of a sharable resource. If several processes attempt to open a read-only file at the same time, they can be granted simultaneous access to the file.
- **Hold and Wait:** To ensure that the hold-and-wait condition never occurs in the system, we must guarantee that, whenever a process requests a resource, it does not hold any other resources.
- One protocol that we can use requires each process to request and be allocated all its resources before it begins execution.
- We can implement this provision by requiring that system calls requesting resources for a process precede all other system calls.
- **No Preemption:** The third necessary condition for deadlocks is that there be no preemption of resources that have already been allocated.
- To ensure that this condition does not hold, we can use the following protocol.
- If a process is holding some resources and requests another resource that cannot be immediately allocated to it (that is, the process must wait), then all resources the process is currently holding are pre-empted.
- **Circular Wait:** One way to ensure that this condition never holds is to impose a total ordering of all resource types and to require that each process requests resources in an increasing order of enumeration.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- We assign a unique integer number to each resource type, which allows us to compare two resources and to determine whether one precedes another in our ordering.
- Formally, we define a one-to-one function $F: R \rightarrow N$

  F(tape drive) = 1
  F(disk drive) = 5
  F(printer) = 12
- That is, a process can initially request any number of instances of a resource type
- —say, Ri . After that, the process can request instances of resource type Rj if and only if $F(Rj) > F(Ri)$.
- A process that wants to use the tape drive and printer at the same time must first request the tape drive and then request the printer.

Que 3.22

Write the Banker's algorithm and how it can be used to avoid can be used to avoid deadlock.

OR

Describe Banker's algorithm for safe allocation.

AKTU 2016-17, Marks 7.5

OR

Write and explain Banker's algorithm for avoidance of deadlock.

AKTU 2011-12, Marks 10

Answer:

An alternative method for avoiding deadlocks is to require additional information about how resources are to be requested.

Deadlock avoidance using Bankers Algorithm:

The name was chosen because the algorithm could be used in a banking system to ensure that the bank never allocated its available cash in such a way that it could no longer satisfy the needs of all its customers.

We need the following data structures, where n is the number of processes in the system and m is the number of resource types:

- **Available.** A vector of length **m** indicates the number of available resources of each type. If Available[j] equals k, then k instances of resource type Rj are available.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- **Max.** An n × m matrix defines the maximum demand of each process. If Max[i][j] equals k, then process Pi may request at most k instances of resource type Rj .
- **Allocation.** An n × m matrix defines the number of resources of each type currently allocated to each process. If Allocation[i][j] equals k, then process Pi is currently allocated k instances of resource type Rj .
- **Need.** An n × m matrix indicates the remaining resource need of each process. If Need[i][j] equals k, then process Pi may need k more instances of resource type Rj to complete its task.
- Note that **Need[i][j] = Max[i][j] − Allocation[i][j]**.

Steps of Safety Algorithm

We can now present the algorithm for finding out whether or not a system is in a safe state.

1. Let **Work** and **Finis**h be vectors of length m and n, respectively. Initialize **Work = Available** and **Finish[i] = false** for i = 0, 1, ..., n − 1.
2. **Find an index i** such that both
   a. Finish[i] == false
   b. Needi ≤Work
   If no such i exists, go to step 4.
3. **Work =Work + Allocation$_i$**
   Finish[i] = true
   Go to step 2.
4. **If Finish[i] == true for all i**, then the system is in a safe state.

Que 3.23

What are the two ways in which deadlock can be detected?

OR

Write an algorithm for detection of deadlock in a system having several instances of multiple resource types.

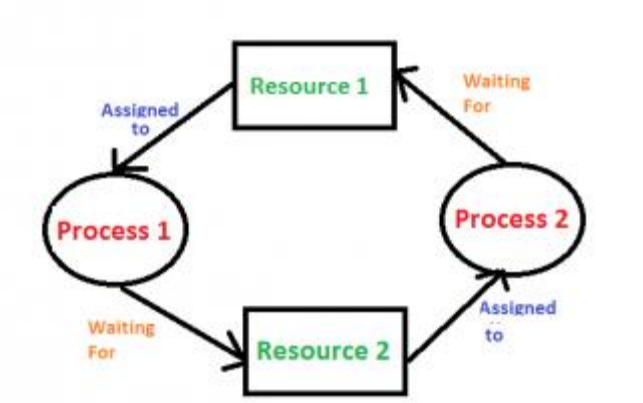
AKTU 2012-13, Marks 10

Answer:

1. **If resources have single instance:**
   In this case for Deadlock detection we can run an algorithm to check for cycle in the Resource Allocation Graph. Presence of cycle in the graph is

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

the sufficient condition for deadlock.



In the above diagram, resource 1 and resource 2 have single instances. There is a cycle R1 → P1 → R2 → P2. So, Deadlock is confirmed.

2. **If there are multiple instances of resources:**
   Detection of the cycle is necessary but not sufficient condition for deadlock detection, in this case, the system may or may not be in deadlock varies according to different situations.

We need several time-varying data structures:

Available. A vector of length *m* indicates the number of available resources of each type.
Allocation. An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
Request. An $n \times m$ matrix indicates the current request of each process. If ***Request***[*i*][*j*] equals *k*, then process *Pi* is requesting *k* more instances of resource type *Rj* .

1. Let***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively.
Initialize ***Work*** = ***Available.***
For $i = 0, 1, ..., n–1$, if $\boldsymbol{Allocation}_i \neq 0$, then ***Finish***[*i*] = ***false.*** Otherwise, ***Finish***[*i*] = ***true.***
2. Find an index *i* such that both
a. ***Finish***[*i*] == ***false***
b. ***Request****i* ≤***Work***
If no such *i* exists, go to step 4.
3. ***Work*** =***Work*** + ***Allocation****i*
Finish[*i*] = ***true***
Go to step 2.
4. If ***Finish***[*i*] ==***false*** for some *i*, $0 \leq i < n$, then the system is in a deadlocked state.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Moreover, if *Finish*[*i*] == *false*, then process *Pi* is deadlocked.

Que 3.24

Describe Banker's algorithm for deadlock avoidance. Consider a system with three processes and three resources. The snapshot of a system at time t0 is given below:

Processes	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	2	2	3	3	6	8	7	7	10
P1	2	0	3	4	3	3			
P2	1	2	4	3	4	4			

1) **What is the content of Need matrix.**
2) **Is the system in safe state?** **AKTU 2018-19, AKTU 2017-18**

Answer:

Processes	Allocation **A**			Max **M**			Need **M - A**			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	2	2	3	3	6	8	1	4	5	7	7	10
P1	2	0	3	4	3	3	2	3	0			
P2	1	2	4	3	4	4	2	2	0			

All processes have need less than the available. Therefore they can be executed in safe manner in sequence P1 → P2 → P3

I. Is the current allocation in safe state?
II. Would the following requests be granted in the current state:
   a. Process P1 requests (1, 0, 0)
   b. Process P1 requests (1, 0, 0)

AKTU 2014-15, Marks 10

Answer:

Processes	Allocation **A**			Max **M**			Need **M - A**			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	2	2	3	3	6	8	1	4	5	7	7	10
P1	2	0	3	4	3	3	2	3	0			
P2	1	2	4	3	4	4	2	2	0			

Initially **Work = 7 7 10**

I. **All processes have need less than the available. Therefore they can be executed in safe manner in sequence P1 → P2 → P3**

II. **a.** Process P2 requests (1, 0, 0)
We will pretend that the resources are allocated to P1, and following changes will be there in the matrix below

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Processes	Allocation **A**			Max **M**			Need **M - A**			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	2	2	3	3	6	8	1	4	5	6	7	10
P1	2	0	3	4	3	3	2	3	0			
P2	2	2	4	3	4	4	1	2	0			

All processes have need less than the available. Therefore they can be executed in safe manner in sequence P1 → P2 → P3

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Que 3.25

Consider the following snapshot of the system:

	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

Answer the following questions using the banker's algorithm:

I. What is the content of the matrix need?
II. Is the system in a safe state? If yes then find the safe sequence.
III. If a request from process P1 arrives for (0, 4, 2, 0) can the request be granted immediately?

AKTU 2015-16, Marks 10

Answer:

i) The need matrix can be calculated according to Banker's Algorithm.

Need matrix

	A	B	C	D
P_0	0	0	0	0
P_1	0	7	5	0
P_2	1	0	0	2
P_3	0	0	2	0
P_4	0	6	4	2

(ii) Yes the system is in safe state. This can be seen as below:

Since we have available = (1,5,2,0) which can be allocated to P_0 or P_2 to satisfy the need or request.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

According to safety algorithm,

	A	B	C	D	
Work =	1	5	2	0	(Available)

	A	B	C	D		A	B	C	D	
Need[P_0] =	0	0	0	0	<	1	5	2	0	(available)

Hence, process [P_0] will release the allocated resources.

Work = Work + allocation

= 1 5 2 0 + 0 0 1 2

	A	B	C	D	
=	1	5	3	2	(Available)

This can satisfy the request of process P_2. Since

	A	B	C	D		A	B	C	D	
Need[P_2] :	1	0	0	2	<	1	5	3	2	(Available)

Hence process P_2 will also release the resource. Hence

Work = work + Allocation

A B C D + A B C D

= 1 5 3 2 + 1 3 5 4

= 2 8 8 6 (Available).

This available can satisfy the request process P_3

Hence resources will be allocated to it.

Work = work + Allocation

= A B C D + A B C D

2 8 8 . 6 + 0 6 3 2

	A	B	C	D	
=	2	14	11	8	(Available)

This available resources, can be allocated to process P_4. Since.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

$$\text{Need } [P_4] = \begin{matrix} A & B & C & D \\ 0 & 6 & 4 & 2 \end{matrix} < \begin{matrix} A & B & C & D \\ 2 & 14 & 11 & 8 \end{matrix}$$

Hence after completion process P_4 will also release the resource

$$\text{Work} = \text{Work} + \text{Allocation}$$

$$= \begin{matrix} A & B & C & D \\ 2 & 14 & 11 & 8 \end{matrix} + \begin{matrix} A & B & C & D \\ 0 & 0 & 1 & 4 \end{matrix}$$

$$= \begin{matrix} A & B & C & D. \\ 2 & 14 & 12 & 12 \end{matrix}$$

These resources can be allocated to process P_1

$$\text{Since Need } [P_1] \begin{matrix} A & B & C & D \\ 0 & 7 & 5 & 0 \end{matrix} < \begin{matrix} A & B & C & D. \\ 2 & 14 & 12 & 12 \end{matrix}$$

hence safe sequence will be $(P_0, P_2, P_3, P_4, P_1)$

(ii) Since Process P_1 request for the resources $(0, 4, 2, 0)$

Now the matrices will be

	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P_0	0	0	1	2	0	0	1	2	1	1	0	0
P_1	1	4	2	0	1	7	5	0				
P_2	1	3	5	4	2	3	5	6				
P_3	0	6	3	2	0	6	5	2				
P_4	0	0	1	4	0	6	5	6				

Here Available

$$= \begin{matrix} A & B & C & D \\ 1 & 5 & 2 & 0 \end{matrix} - \begin{matrix} A & B & C & D \\ 0 & 4 & 2 & 0 \end{matrix} = \begin{matrix} A & B & C & D \\ 1 & 1 & 0 & 0 \end{matrix}$$

Now the need matrix can be represented as

$$\text{Need } i = \text{Max}_i - \text{Allocation}_i$$

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

This is the modified table :

	Allocation				Max				Need				Available			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P_0	0	0	1	2	0	0	1	2	0	0	0	0	1	1	0	0
P_1	1	4	2	0	1	7	5	0	0	3	3	0				
P_2	1	3	5	4	2	3	5	6	1	0	0	2				
P_3	0	6	3	2	0	6	5	2	0	0	2	0				
P_4	0	0	1	4	0	6	5	6	0	6	4	2				

The safe sequence still remains the same

The system continues to remain in a safe state.

Que 3.26

In a system, n processes share m resource units that can be reserved and released only one at a time. The maximum need of each process does not exceed m and sum of all maximum need is less than m + n. Show that a deadlock cannot occur.

AKTU 2011-12, Marks 05

Answer:

Consider the worst case- all processes acquire maximum resources but still not able to finish. So, the resources available must be 1 less than the maximum need, for each of the processes (this ensures none of them can finish).

We are given maximum need is always less than m + n. As per our condition for deadlock, resources available must be 1 less than maximum need for each of 'm' processes => totally the resources available must be less than m + n - m = n.

But 'n' is the available number of resources and hence no deadlock can occur.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Que 3.27

Describe the Banker's algorithm for safe allocation. Consider a system with five processes and three resource types and at time T0 the following snapshot of the system has been taken:

AKTU 2013-14, Marks 10

Process id	Allocated			Maximum			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	1	1	2	4	3	3	3	1	0
P2	2	1	2	3	2	2			
P3	4	0	1	9	0	2			
P4	0	2	0	7	5	3			
P5	1	1	2	11	2	3			

I. Determine the total amount of resources of each type.

Total no of resources = Total allocated + Available

For R1 = 8 + 3 = 11

For R2 = 5 + 1 = 6

For R3 = 7 + 0 = 7

II. Compute the need matrix.

Process id	Allocated			Maximum			**Need** Maximum – Allocated		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	1	1	2	4	3	3	3	2	1
P2	2	1	2	3	2	2	1	1	0
P3	4	0	1	9	0	2	5	0	1
P4	0	2	0	7	5	3	7	3	3
P5	1	1	2	11	2	3	10	1	1

III. Determine if the state is safe or not using Banker's algorithm.

Work = Available

Work = 3 1 0

For P1 need = 3 2 1 > 3 1 0

(which is greater than the work, hence not able to fulfill.)

For P2 Need = 1 1 0 < 3 1 0 **(resources can be allocated)**

Work = work + Allocation

= 3 1 0

+ 2 1 2

Work = 5 2 2

For P3 Need = 5 0 1 < 5 2 2 **(resources can be allocated)**

Work = work + Allocation

= 5 2 2

+ 4 0 1

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Work = 9 2 3

For P4 Need = 7 3 3 > 9 2 3 (which is greater than the work, hence not able to fulfill.)

For P5 Need = 10 1 1 > 9 2 3 (which is greater than the work, hence not able to fulfill.)

Now we will again go for P1

For P1 need = 3 2 1 < 9 2 3 **(resources can be allocated)**

Work = work + Allocation
= 9 2 3
+ 1 1 2
Work = 10 3 5

For P4 Need = 7 3 3 < 10 3 5 **(resources can be allocated)**

Work = work + Allocation
= 10 3 5
+ 0 2 0
Work = 10 5 5

For P5 Need = 10 1 1 < 10 5 5 **(resources can be allocated)**

So there is a safe sequence P2 →P3 →P1 →P4 →P5
Therefore, state is safe.

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Numerical :

(i) Total resource = Total allocated (sum of column of allocation + available)

$= [8\ 5\ 7] + [3\ 1\ 0] = [11\ 6\ 7]$

(ii) Need = Maximum − Allocation

A B C	−	A B C		Need
4 3 3		1 1 2	=	3 2 1
3 2 2		2 1 2	=	1 1 0
9 0 2		4 0 1	=	5 0 1
7 5 3		0 2 0	=	7 3 3
11 2 3		1 1 2	=	10 1 1

(iii) Need is compared with available. If need ≤ available, then resources are allocated to that process and process will release the resource

If need is greater than Available, next process need is taken for comparision. Need for process P_1 is (3 2 1) and Available is (3 1 0)

Need > Available → False.

So system will move to next process.

Need for P_2 is (1 1 0) and available is (3 1 0)

Need ≤ Available → True

Request for P_2 is granted.

Available = Available + Allocation

$= 310 + 212 = 522$

Que 3.28

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

Differentiate between deadlock and starvation.

AKTU 2011-12, Marks 2.5

Answer:

DEADLOCK	STARVATION
All processes keep waiting for each other to complete and none get executed	High priority processes keep executing and low priority processes are blocked
Resources are blocked by the processes	Resources are continuously utilized by high priority processes
Necessary conditions Mutual Exclusion, Hold and wait, No pre-emption, circular wait.	Priorities are assigned to the processes
It can be prevented by avoiding necessary conditions for deadlock	It can be prevented by aging.

Que 3.29

Discuss the techniques to recover from deadlock.

Answer

1. Process Termination:

To eliminate the deadlock, we can simply kill one or more processes. For this, we use two methods:

- **(a). Abort all the Deadlocked Processes:**
  Aborting all the processes will certainly break the deadlock, but with a great expenses. The deadlocked processes may have computed for a long time and the result of those partial computations must be discarded and there is a probability to recalculate them later.

- **(b). Abort one process at a time until deadlock is eliminated:**
  Abort one deadlocked process at a time, until deadlock cycle is eliminated from the system. Due to this method, there may be considerable overhead, because after aborting each process, we have to run deadlock detection algorithm to check whether any processes are still deadlocked.

2. Resource Preemption:

To eliminate deadlocks using resource preemption, we preempt some resources from processes and give those resources to other processes. This method will raise three issues –

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)

- **(a). Selecting a victim:**
  We must determine which resources and which processes are to be preempted and also the order to minimize the cost.
- **(b). Rollback:**
  We must determine what should be done with the process from which resources are preempted. One simple idea is total rollback. That means abort the process and restart it.
- **(c). Starvation:**
  In a system, it may happen that same process is always picked as a victim. As a result, that process will never complete its designated task. This situation is called **Starvation** and must be avoided. One solution is that a process must be picked as a victim only a finite number of times

Downloaded by Riddhi Jaiswal (riddhi9892jaiswal@gmail.com)